AI Critique

Text Prompt

We have attached our use case diagram and our functional and non-functional requirements. Please give us an evaluation on whether our diagram matches our requirements.

Original ChatGPT Answer

Below is a structured evaluation of whether your **use case diagram aligns with your functional and non-functional requirements**.

---

1. Overall Assessment

Short answer:

Your use case diagram captures the *core functional flow* well (trip input → risk assessment → route generation → ranking → display).

However, it:

- ✅ Covers most high-level functional capabilities
- ⚠️ Omits or under-represents several detailed requirements
- ❌ Does not reflect most Non-Functional Requirements (which is partly acceptable, but some should be implied structurally)

The diagram is **conceptually aligned**, but it is **not fully complete relative to the detailed specification**.

---

2. Strong Alignment Areas (Well Reflected in Diagram)

A. Preference Management

Your diagram includes:

- New Preference
- Modify Preference
- Display Preferences
- Save Preference
- Fetch Preference

These match:

- 1.1 First-time detection (partially implied)
- 1.2 Initial preference input
- 1.7 Saving preferences
- 2.1 Retrieval of saved default profile
- 2.2 Modify preferences for this trip

✅ Good alignment at a high level.

B. Trip Input Handling

Diagram includes:

- Initiate Trip
- Input Origin

- Input Destination
- Input Departure Time
- Validate Input
- Fetch Suggestions
- Select Location from Map
- Use Current Location
- Request Location Permission
- Confirm Trip Request

These align well with:

- 2.4 Origin input (GPS + manual + validation)
- 2.5 Destination input
- Departure time rules
- Suggestion updates
- Permission handling

✅ Very strong mapping here.

C. Risk Assessment & Data Retrieval

Diagram includes:

- Retrieve Real-Time Data
- Assess Delay Risk
- Assess Crowding Risk
- Generate Scores

These match:

- Section 3 (Assess Transport Conditions and Generate Risk Scores)
- Segment-level delay risk
- Segment-level crowding risk

✅ Conceptually aligned.

D. Route Generation

Diagram includes:

- Generate Routes
- Generate Scores
- Fetch Preference
- Refresh Routes

These reflect:

- Section 4 Route Generation
- Ranking according to preference
- Refresh functionality

✅ Core pipeline is represented.

---

3. Major Gaps Between Diagram and Requirements

Now the important part.

❌ **1. First-Time User Detection Not Explicit**

Requirement:
1.1.1 The system shall detect first-time access
Your diagram:

- Shows “New Preference”
- Does NOT explicitly show:
    - Detect First-Time User
    - Conditional prompt

📌 Recommendation:
Add a use case:

- **Detect First-Time User**
    - <> Prompt Initial Preference Setup

❌ 2. Mode Constraints & Hybrid Taxi Not Represented

Your requirements contain detailed transport mode rules:

- Enable/disable MRT-only, MRT+Bus, Taxi, Drive
- Hybrid Taxi constraints
- Maximum taxi segment duration
- Maximum taxi budget
- Public transport only restriction

Your diagram:

- Does NOT show:
    - Configure Transport Modes
    - Hybrid Taxi option
    - Budget constraint
    - Taxi duration constraint

📌 These are major functional requirements but invisible in the diagram.
Recommendation:
Add use cases such as:

- Configure Transport Modes
- Configure Hybrid Taxi Constraints
- Configure Budget Constraint
- Configure Walking Tolerance
- Configure Transfer Tolerance

❌ 3. Route Ranking Logic Not Explicit

Your requirements contain:

- Ranking by Fastest
- Ranking by Most Reliable
- Ranking by Most Comfortable
- Ranking by Most Economical
- Tie-breaking rules

Diagram only shows:

- Generate Routes
- Generate Scores

It does NOT show:

- Rank Routes
- Apply Optimisation Strategy

📌 Ranking is central to your system and should be a clear use case.

Recommendation:

Add:

- **Rank Routes**
  - <> Apply Optimisation Strategy

❌ 4. Route Explanation Missing

Requirement:

4.8 The system shall provide a one-line explanation

Your diagram:

- Does NOT show:
  - Generate Route Explanation
  - Display Route Justification

Recommendation:

Add:

- Generate Route Explanation
  - <> Explain Alignment with Preference

❌ 5. Data Freshness & Fallback Not Represented

Section 5 is detailed and important:

- Validate freshness
- Apply fallback data
- Prevent mixing data
- Mark fallback routes
- Show timestamp
- Warning if older than 15 minutes

Your diagram:

- Only shows Retrieve Real-Time Data
- Does NOT show:
  - Validate Data Freshness
  - Apply Fallback Data
  - Mark Fallback Mode
  - Display Timestamp
  - Show Warning

This is a major architectural behavior missing from diagram.

Recommendation:

Add:

- Validate Data Freshness
- Apply Fallback Data
- Mark Route with Fallback Warning

❌ 6. Settings & Localisation Incomplete

Diagram shows:

- Change Settings
- Save Settings

But your requirements include:

- Language selection
- Unit format selection
- Reset to default
- View saved preference profile
- Update saved preference profile

These are not decomposed in diagram.

Recommendation:

Expand Settings into:

- Change Language
- Change Unit Format
- Reset Settings
- Update Default Profile

❌ 7. Non-Functional Requirements Not Reflected

While NFRs are not usually shown directly in use case diagrams, some architectural aspects could be implied:

Missing structural hints for:

- Support 200 concurrent users
- 5-second response time
- Refresh interval ≤ 5 min
- Cached data usage
- Availability when data source fails

These are not expected as use cases, but your fallback mechanism should be more visible because it affects behavior.

---

4. Structural Observations About the Diagram

A. Overuse of <> and <>

There are many include/extend relationships, which makes the diagram:

- Hard to read
- Slightly over-engineered

Example:

- Input Origin <> Fetch Suggestions
- Input Origin <> Select Location from Map

This is technically correct, but you could simplify:

Group into higher-level use cases:

- Enter Origin
- Enter Destination

Instead of showing every micro-step.

B. Missing Actor Roles

You only have:

- User

But logically, you also interact with:

- External Data Providers (MRT API, Taxi API, Carpark API)

Even if not required, adding:

- External Transport Data Service (as secondary actor)

would improve completeness.

---

5. Summary Evaluation

Area	Alignment
Preference Setup	✅ Mostly covered
Trip Input	✅ Strong alignment
Risk Assessment	✅ Conceptually covered
Route Generation	✅ Covered at high level
Ranking Logic	❌ Not explicit
Mode Constraints	❌ Missing
Hybrid Taxi	❌ Missing
Budget / Walking / Transfers	❌ Missing
Data Freshness & Fallback	❌ Major gap
Route Explanation	❌ Missing
Settings & Localisation	⚠️ Partial
Non-Functional Representation	⚠️ Minimal

Final Verdict

Your use case diagram:

✓ Correctly represents the main system workflow

✓ Covers the primary user journey

❌ Does not fully reflect the depth of your functional specification

❌ Misses several important constraint-based behaviors

If graded academically, I would estimate:

Alignment Score: 7 / 10

It is strong structurally, but incomplete relative to your very detailed requirements.

Recommended Improvements (High Priority)

1. Add explicit:
    - Rank Routes
    - Apply Optimisation Strategy
    - Generate Route Explanation
2. Add constraint-related use cases:
    - Configure Transport Modes
    - Configure Hybrid Taxi Constraints

    - Configure Budget / Walking / Transfer Tolerance
3. Add:
    - Validate Data Freshness
    - Apply Fallback Handling
    - Mark Fallback Warning
4. Refactor includes/extends to reduce visual clutter.

If you'd like, I can:

- Redesign a cleaner improved version of your use case structure
- Or provide a marked-up correction plan suitable for submission

Just tell me your academic level (undergrad / masters / capstone / FYP).

Selected Quote

"Your use case diagram correctly represents the main system workflow, but it does not fully reflect the depth of your functional specification and misses several important constraint-based behaviours."

Justification

We selected this aspect as it highlights our most critical issue: the difference between capturing high-level workflow and fully modelling detailed functional requirements. While our use case diagram appears structurally correct, ChatGPT emphasised that many constraint-based behaviours such as hybrid taxi limits, budget constraints and fallback data handling were not explicitly represented. This distinction is crucial in system design as detailed requirements significantly impact system behaviour and correctness.